动态维护中位数
并查集
树状数组
二维树状数组
线段树（区间加，区间查询）
线段树合并
树链剖分
笛卡尔树
倍增求LCA
主席树
主席树求静态区间第k小
分块
二进制分块
珂朵莉树
数位dp

数据结构

动态维护中位数

```
struct Median
{
    std::priority_queue<int> L;
    std::priority_queue<int,std::vector<int>,std::greater<int>> R;
    int sl = 0, sr = 0;
    void add(int x)
    {
        if(L.empty() || x <= L.top())
        {
            L.push(x);
            sl += x;
        }
        else
        {
            R.push(x);
            sr += x;
        }

        while(L.size() > R.size() + 1)
        {
            int x = L.top();
            L.pop();
            sl -= x;

            R.push(x);
            sr += x;
        }

        while(L.size() < R.size())
```

```
        {
            int x = R.top();
            R.pop();
            sr -= x;

            L.push(x);
            sl += x;
        }
    }
    int query()
    {
        if(L.empty())return 0;
        int mid = L.top();
        return mid * L.size() - sl + sr - mid * R.size();
    }
};
```

并查集

```
struct dsu
{
    int n;
    std::vector<int>fa,siz;
    dsu(int n): n(n),fa(n + 1),siz(n + 1,1)
    {
        for(int i = 1;i <= n;i++)
            fa[i] = i;
    }
    int find(int x)
    {
        if(x == fa[x])return x;
        return fa[x] = find(fa[x]);
    }
    bool merge(int a,int b)
    {
        a = find(a);
        b = find(b);
        if(a == b)return false;
        if(siz[a] < siz[b])std::swap(a,b);
        siz[a] += siz[b];
        fa[b] = a;
        return true;
    }
};
```

树状数组

```
struct finwick
{
```

```
    int n;
    std::vector<int>t;
    finwick(int n): n(n),t(n + 1){}
    int lowbit(int x)
    {
        return x & (-x);
    }
    void add(int x,int k)
    {
        for(int i = x;i <= n;i += lowbit(i))
            t[i] += k;
    }
    int query(int l,int r)
    {
        if(l > r)return 0;
        int ans = 0;
        for(int i = r;i >= 1;i -= lowbit(i))
            ans += t[i];
        for(int i = l - 1;i >= 1;i -= lowbit(i))
            ans -= t[i];
        return ans;
    }
};
```

二维树状数组

```
struct finwick2
{
    int n,m;
    std::vector<std::vector<int>>t;
    finwick2(int n,int m): n(n),m(m),t(n + 1,std::vector<int>(m + 1)){}
    int lowbit(int x)
    {
        return x & -x;
    }
    void add(int x,int y,int k)
    {
        for(int i = x;i <= n;i += lowbit(i))
            for(int j = y;j <= m;j += lowbit(j))
                t[i][j] += k;
    }
    int query(int x,int y)
    {
        int res = 0;
        for(int i = x;i >= 1;i -= lowbit(i))
            for(int j = y;j >= 1;j -= lowbit(j))
                res += t[i][j];
        return res;
    }
    int query(int r1,int c1,int r2,int c2)
    {
        if(r1 > r2 || c1 > c2)return 0;
```

```
        return query(r2,c2) - query(r2,c1 - 1) - query(r1 - 1,c2) + query(r1 - 1,c1 - 1);
    }
};
```

线段树（区间加，区间查询）

```
struct seg
{
    struct node
    {
        int l;
        int r;
        int sum;
        int lz;
    };
    std::vector<node>t;
    void push_up(int p)
    {
        t[p].sum = (t[p << 1].sum + t[p << 1 | 1].sum);
    }
    void push_down(int p)
    {
        if(t[p].lz)
        {
            t[p << 1].sum += (t[p << 1].r - t[p << 1].l + 1) * t[p].lz;
            t[p << 1].lz += t[p].lz;
            t[p << 1 | 1].sum += (t[p << 1 | 1].r - t[p << 1 | 1].l + 1) * t[p].lz;
            t[p << 1 | 1].lz += t[p].lz;
            t[p].lz = 0;
        }
    }
    void build(int p,int l,int r)
    {
        t[p] = {l,r,0,0};
        if(l == r)
            return;
        int mid = (l + r) >> 1;
        build(p << 1,l,mid);
        build(p << 1 | 1,mid + 1,r);
        push_up(p);
    }
    seg(int n)
    {
        t.resize((n << 2) + 7);
        build(1,1,n);
    }
    void update(int p,int nl,int nr,int k)
    {
        if(nl > nr)return;
```

```
        if(nl <= t[p].l && t[p].r <= nr)
        {
            t[p].sum += (t[p].r - t[p].l + 1) * k;
            t[p].lz += k;
            return;
        }
        push_down(p);
        int mid = (t[p].l + t[p].r) >> 1;
        if(nl <= mid)update(p << 1,nl,nr,k);
        if(nr > mid)update(p << 1 | 1,nl,nr,k);
        push_up(p);
    }
    int query(int p,int nl,int nr)
    {
        if(nl > nr)return 0;
        if(nl <= t[p].l && t[p].r <= nr)
            return t[p].sum;
        push_down(p);
        int mid = (t[p].l + t[p].r) >> 1,res = 0;
        if(nl <= mid)res += query(p << 1,nl,nr);
        if(nr > mid)res += query(p << 1 | 1,nl,nr);
        return res;
    }
};

```

线段树合并

```
struct node//维护1 ~ up 最大数量及其编号
{
    int l;
    int r;
    int mxsum;
    int mxpos;
}t[N << 6];
int rt[N],tot = 0,up = 1e5;
void push_up(int p)
{
    int ls = t[p].l,rs = t[p].r;
    if(t[ls].mxsum >= t[rs].mxsum)
    {
        t[p].mxpos = t[ls].mxpos;
        t[p].mxsum = t[ls].mxsum;
    }
    else
    {
        t[p].mxpos = t[rs].mxpos;
        t[p].mxsum = t[rs].mxsum;
    }
}
```

```
int add(int id,int l,int r,int pos,int k)
{
    if(!id)id = ++tot;
    if(l == r)
    {
        t[id].mxsum += k;
        t[id].mxpos = pos;
        return id;
    }
    int mid = (l + r) >> 1;
    if(pos <= mid)t[id].l = add(t[id].l,l,mid,pos,k);
    else t[id].r = add(t[id].r,mid + 1,r,pos,k);
    push_up(id);
    return id;
}
int merge(int a,int b,int l,int r)
{
    if(!a)return b;
    if(!b)return a;
    if(l == r)
    {
        t[a].mxsum += t[b].mxsum;
        return a;
    }
    int mid = (l + r) >> 1;
    t[a].l = merge(t[a].l,t[b].l,l,mid);
    t[a].r = merge(t[a].r,t[b].r,mid + 1,r);
    push_up(a);
    return a;
}
```

树链剖分

```
const int N = 2e5+7;
struct node
{
    int l;
    int r;
    int w;
    int lz = 0;
}t[N << 2];
int a[N],wt[N];
int n, m, root, P;
int dep[N], son[N], id[N], fa[N], top[N], siz[N], tot = 0;
struct edge
{
    int nx;
    int to;
}e[N << 1];
int heads[N], cntqq = 0;
void add(int u, int v)
{
```

```
    e[++cntqq].nx = heads[u];
    e[cntqq].to = v;
    heads[u] = cntqq;
    return;
}
void push_up(int p)
{
    t[p].w = (t[p << 1].w + t[p << 1 | 1].w)%P;
    return;
}
void push_down(int p)
{
    t[p << 1].lz = (t[p << 1].lz + t[p].lz);
    t[p << 1 | 1].lz = (t[p << 1 | 1].lz + t[p].lz);
    int l1 = t[p << 1].r - t[p << 1].l + 1;
    int l2 = t[p << 1|1].r - t[p << 1|1].l + 1;
    t[p << 1].w = (t[p << 1].w + l1 * t[p].lz) % P;
    t[p << 1 | 1].w = (t[p << 1 | 1].w + l2 * t[p].lz) % P;
    t[p].lz = 0;
    return;
}
inline void build(int p, int l, int r)
{
    t[p].l = l, t[p].r = r;
    if (l == r)
    {
        t[p].w = wt[l] % P;
        return;
    }
    int mid = (l + r) >> 1;
    build(p << 1, l, mid);
    build(p << 1 | 1, mid + 1, r);
    push_up(p);
}
inline void update(int p, int nl, int nr, int ww)
{

    if (nl <= t[p].l && t[p].r <= nr)
    {
        t[p].lz = (t[p].lz + ww);
        t[p].w = (t[p].w + ww * (t[p].r - t[p].l + 1))%P;
        return;
    }
    push_down(p);
    int mid = (t[p].l + t[p].r) >> 1;
    if (nl <= mid)update(p << 1, nl, nr, ww);
    if (nr > mid)update(p << 1 | 1, nl, nr, ww);
    push_up(p);
}
inline int query(int p, int nl, int nr)
{

    if (nl <= t[p].l && t[p].r <= nr)
```

```
    {
        return t[p].w % P;
    }
    push_down(p);
    int re = 0;
    int mid = (t[p].l + t[p].r) >> 1;
    if (nl <= mid)re = (re + query(p << 1, nl, nr)) % P;
    if(nr > mid)re = (re + query(p << 1 | 1, nl, nr)) % P;
    return re;
}
inline int qRange(int x, int y)
{
    int ans = 0;
    while (top[x] != top[y])
    {
        if (dep[top[x]] < dep[top[y]])swap(x, y);
        ans = (ans + query(1, id[top[x]], id[x])) % P;
        x = fa[top[x]];
    }
    if (dep[x] > dep[y])swap(x, y);
    ans = (ans + query(1, id[x], id[y])) % P;
    return ans;
}
inline void upRange(int x, int y, int k)
{
    k = k % P;
    while (top[x] != top[y])
    {
        if (dep[top[x]] < dep[top[y]])swap(x, y);
        update(1, id[top[x]], id[x], k);
        x = fa[top[x]];
    }
    if (dep[x] > dep[y])swap(x, y);
    update(1, id[x], id[y], k);
}
inline int qSon(int x)
{
    return query(1, id[x], id[x] + siz[x] - 1);
}
inline void upSon(int x, int k)
{
    update(1, id[x], id[x] + siz[x] - 1, k);
}


inline void dfs1(int p, int f, int deep)//深度，父亲，子树大小，重儿子编号
{
    dep[p] = deep;
    fa[p] = f;
    siz[p] = 1;
    int zson = -1;
    for (int i = heads[p]; i; i = e[i].nx)
    {
```

```
        int v = e[i].to;
        if (v == f)continue;
        dfs1(v, p, deep + 1);
        siz[p] += siz[v];
        if (siz[v] > zson)son[p] = v, zson = siz[v];
    }
}
inline void dfs2(int p, int topf)//top,dfs序
{
    id[p] = ++tot;
    wt[tot] = a[p];
    top[p] = topf;
    if (!son[p])return;
    dfs2(son[p], topf);
    for (int i = heads[p]; i; i = e[i].nx)
    {
        int v = e[i].to;
        if (v == fa[p] || v == son[p])continue;
        dfs2(v, v);
    }
}
inline void solve()
{
    cin >> n >> m >> root >> P;
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    for (int i = 1; i <= n - 1; i++)
    {
        int u, v;
        cin >> u >> v;
        add(u, v); add(v, u);
    }
    dfs1(root, 0, 1);
    dfs2(root, root);
    build(1, 1, n);
    while (m--)
    {
        int k, x, y, z;
        cin >> k;
        switch (k)
        {
        case 1:
            cin >> x >> y >> z;
            upRange(x, y, z);
            break;
        case 2:
            cin >> x >> y;
            cout << qRange(x, y)<<endl;
            break;
        case 3:
            cin >> x >> z;
            upSon(x, z);
            break;
```

```
        case 4:
            cin >> x;
            cout << qSon(x) << endl;
        }
    }
}
```

笛卡尔树

```
//求连续区间最小值(也可以维护最大值)
vector<int>w(n + 1);
vector<int>ls(n + 1,0),rs(n + 1,0);//左子树 右子树
stack<int>stk;
for(int i = 1;i <= n;i++)
{
    int le = -1;//弹出链的根节点
    while(!stk.empty() && w[stk.top()] > w[i]){le = stk.top();stk.pop();}
    //把右链上大于w[i]的部分弹出来
    if(!stk.empty())rs[stk.top()] = i;//i链接到右链上
    if(le != -1)ls[i] = le;//弹出的链变成i的左子树
    stk.push(i);
}
//栈底为根节点
```

倍增求LCA

```
struct Graph
{
    struct edge
    {
        int nx;
        int to;
        int w;
    };

    std::vector<edge> e;
    std::vector<int> heads;

    Graph(int n = 0,int m = 0) : heads(n + 1,-1)
    {
        e.reserve(m);
    }

    void add(int u,int v,int w = 0)
    {
        e.push_back({heads[u],v,w});
        heads[u] = e.size() - 1;
    }
};
struct LCA : Graph
{
```

```cpp
    int root,n,LOG;
    std::vector<std::vector<int>>fa;
    std::vector<int>dep;

    LCA(int num,int rt = 1) : Graph(num,2 * std::max(0LL,num - 1))
    {
        n = num;
        root = rt;
        LOG = 1;
        while((1LL << LOG) <= n)LOG++;

        fa.assign(n + 1,std::vector<int>(LOG));
        dep.assign(n + 1,0);
    }

    void init(int u)
    {
        for(int i = 1;i < LOG;i++)
            fa[u][i] = fa[fa[u][i - 1]][i - 1];

        for(int i = heads[u];~i;i = e[i].nx)
        {
            int v = e[i].to;
            if(v == fa[u][0])continue;
            fa[v][0] = u;
            dep[v] = dep[u] + 1;
            init(v);
        }
    }
    void build()
    {
        init(root);
    }
    int lca(int x,int y)
    {
        if(dep[x] < dep[y])std::swap(x,y);
        int d = dep[x] - dep[y];
        for(int i = 0;d;i++)
        {
            if(d & 1)x = fa[x][i];
            d >>= 1;
        }
        if(x == y)return x;
        for(int i = LOG - 1;i >= 0;i--)
        {
            if(fa[x][i] != fa[y][i])
            {
                x = fa[x][i];
                y = fa[y][i];
            }
        }
        return fa[x][0];
    }
```

```
};
void solve()
{
    int n,rt;
    std::cin >> n >> rt;
    LCA g(n,rt);
    for(int i = 1;i < n;i++)
    {
        int u,v;
        std::cin >> u >> v;
        g.add(u,v);
        g.add(v,u);
    }
    g.build();
    int x,y;
    std::cin >> x >> y;
    std::cout << g.lca(x,y) << "\n";
}
```

主席树

```
struct node
{
    int l;
    int r;
    int w;
}t[N << 5];
int a[N],tot = 0;
int cln(int p)//克隆
{
    t[++tot] = t[p];
    return tot;
}
int build(int l,int r)//建初始树
{
    int p = ++tot;
    if(l == r)
    {
        t[p].w = a[l];
        return p;
    }
    int mid = (l + r) >> 1;
    t[p].l = build(l,mid);
    t[p].r = build(mid + 1,r);
    return p;
}
int update(int p,int l,int r,int pos,int w)//返回复制的新节点
{
    int cl = cln(p);
    if(l == r)t[cl].w = w;
    else
    {
```

```
        int mid = (l + r) >> 1;
        if(pos <= mid)t[cl].l = update(t[p].l,l,mid,pos,w);
        else t[cl].r = update(t[p].r,mid + 1,r,pos,w);
    }
    return cl;
}
int query(int p,int l,int r,int pos)//查询
{
    if(l == r)return t[p].w;
    int mid = (l + r) >> 1;
    if(pos <= mid)return query(t[p].l,l,mid,pos);
    else return query(t[p].r,mid + 1,r,pos);
}
//使用方法
rt[M];//rt[i]为版本i的根节点
rt[0] = build(1,n);//可以不需要初始建树
rt[i] = update(rt[v],1,n,pos,w)//在版本v上将下标为pos的值更新为w，并返回新版本的根节点下标
query(rt[v],1,n,pos)//询问v版本下标为pos的值
```

主席树求静态区间第k小

```
struct node
{
    int l;
    int r;
    int cnt;
}t[N << 5];int tot = 0;
int cln(int p)
{
    t[++tot] = t[p];
    return tot;
}
int build(int l,int r)
{
    int p = ++tot;
    if(l == r)return p;
    int mid = (l + r) >> 1;
    t[p].l = build(l,mid);
    t[p].r = build(mid + 1,r);
    return p;
}
int update(int p,int l,int r,int pos)
{
    int cl = cln(p);t[cl].cnt++;
    if(l == r)return cl;
    int mid = (l + r) >> 1;
    if(pos <= mid)t[cl].l = update(t[p].l,l,mid,pos);
    else t[cl].r = update(t[p].r,mid + 1,r,pos);
    return cl;
};
int query(int u,int v,int l,int r,int k)
{
```

```
    if(l == r)return l;
    int mid = (l + r) >> 1;
    int tmp = t[t[v].l].cnt - t[t[u].l].cnt;
    if(k <= tmp)return query(t[u].l,t[v].l,l,mid,k);
    else return query(t[u].r,t[v].r,mid + 1,r,k - tmp);
}
void solve()
{
    int n,m;
    cin >> n >> m;
    vector<int>a(n + 1),b(n + 1),rt(n + 1);
    for(int i = 1;i <= n;i++)
    {
        cin >> a[i];
        b[i] = a[i];
    }
    //离散化
    sort(b.begin() + 1,b.end());
    b.erase(unique(b.begin() + 1,b.end()),b.end());
    int len = b.size();
    auto get_pos = [&](int w) -> int
    {
        return lower_bound(b.begin() + 1,b.end(),w) - b.begin();
    };
    rt[0] = build(1,len);
    for(int i = 1;i <= n;i++)
        rt[i] = update(rt[i - 1],1,len,get_pos(a[i]));
    while(m--)
    {
        int l,r,k;
        cin >> l >> r >> k;
        cout << b[query(rt[l - 1],rt[r],1,len,k)] << endl;
    }
}
```

分块

```
void solve()
{
    int n,m,u;
    cin >> n >> m >> u;
    int len = sqrt(n) + 1;//块长
    vector<int>a(n + 1),id(n + 1);//原数组，编号
    vector<vector<int>>b(len + 1);//块内数
    for(int i = 1;i <= n;i++)
    {
        cin >> a[i];
        id[i] = (i - 1) / len + 1;
        b[id[i]].push_back(a[i]);
    }
    for(int i = 1;i <= len;i++)sort(b[i].begin(),b[i].end());
    auto query = [&](int l,int r,int v) -> int
```

```
    {
        int lid = id[l],rid = id[r];
        int sum = 0;
        if(lid == rid)
        {
            for(int i = l;i <= r;i++)
                if(a[i] < v)sum++;
            return sum;
        }
        for(int i = l;id[i] == lid;i++)
            if(a[i] < v)sum++;
        for(int i = lid + 1;i < rid;i++)
            sum += lower_bound(b[i].begin(),b[i].end(),v) - b[i].begin();
        for(int i = r;id[i] == rid;i--)
            if(a[i] < v)sum++;
        return sum;
    };
    auto update = [&](int pos,int k,int l,int r) -> void
    {
        int iid = id[pos];
        b[iid].erase(lower_bound(b[iid].begin(),b[iid].end(),a[pos]));
        a[pos] = u * k / (r - l + 1);
        b[iid].push_back(a[pos]);
        sort(b[iid].begin(),b[iid].end());
    };
    while(m--)
    {
        int l,r,v,p;
        cin >> l >> r >> v >> p;
        int cnt = query(l,r,v);
        update(p,cnt,l,r);
    }
    for(int i = 1;i <= n;i++)
        cout << a[i] << endl;
}
```

二进制分块

可以将一个二进制数组按64位分块存储

一般用于查询时，要选择子数组的二进制并对其进行位运算操作

```
typedef unsigned long long ull;
ull A[N];
ull Bit(int l,int r)
{
    ull ret = (r == 63) ? -1ull : (1ull << (r + 1)) - 1;
    ret ^= (1ull << l) - 1;
    return ret;
}
struct Bitset
{
    vector<ull>arr;//按64分块存储
```

```
    int pos;
    int sz;
    Bitset(int _sz) {
        sz = _sz; // 位总长度
        pos = 0; // 当前添加位置
        arr.resize((_sz + 63) >> 6); // 按64位分块
    }
    void add(int cnt,ull val)//将长度为cnt的val加进来
    {
        if(cnt <= 64 - (pos & 63))
            arr[pos >> 6] |= val << (pos & 63);
        else
        {
            ull mask = Bit(0,64 - (pos & 63) - 1);
            arr[pos >> 6] |= (val & mask) << (pos & 63);
            arr[(pos >> 6) + 1] = val >> (64 - (pos & 63));
        }
        pos += cnt;
    }
    ull calc()//对于每一块进行答案统计
    {
        ull ret = 0;
        int ra = 0;
        for(int i = 0;i < arr.size();i++)
        {

        }
        return ret;
    }
};
Bitset getBitset(int l,int r)//将原数组[l,r]取出来
{
    Bitset res(r - l + 1);
    if((l >> 6) == (r >> 6))
    {
        ull val = (A[l >> 6] & Bit(l & 63,r & 63)) >> (l & 63);
        res.add(r - l + 1,val);
    }
    else
    {
        ull val = (A[l >> 6] & Bit(l & 63,63)) >> (l & 63);
        res.add(63 - (l & 63) + 1,val);
        for(int i = (l >> 6) + 1;i < (r >> 6);i++)
            res.add(64,A[i]);
        val = A[r >> 6] & Bit(0,r & 63);
        res.add((r & 63) + 1,val);
    }
    return res;
}

void solve()
{
    //初始化方法
```

```
    string s;cin >> s;
    for(int i = 0;i < n;i += 64)
    {
        ull val = 0;
        for(int j = 0;j < 64 && j + i < n;j++)
            val |= (ull)(s[i + j] - '0') << j;
        A[i >> 6] = val;
    }
}
```

珂朵莉树

```
//将值相同的一段区间合并进行维护
//每次遍历后更新区间(区间被合并成一个节点),时间复杂度mlog(n),否则复杂度容易爆
//也可以用于保证数据随机的题目
struct Chtholly_Tree
{
    struct node
    {
        int l;
        int r;
        mutable int v;//可直接操作
        bool operator<(const node &tp)const{
            return l < tp.l;
        }
    };
    set<node>s;
    Chtholly_Tree(int n)
    {
        s.insert({1,n + 1,0});
    }
    auto spilt(int x)//将[l,r]分裂成[l,x),[x,r]
    {
        auto it = s.lower_bound(node{x,0,0});
        if(it != s.end() && it->l == x)return it;
        --it;
        int l = it->l,r = it->r,v = it->v;
        s.erase(it);
        s.insert({l,x - 1,v});
        return s.insert({x,r,v}).first;
    }
    void assign(int l,int r,int v)//将[l,r]赋值为v
    {
        auto itr = spilt(r + 1),itl = spilt(l);
        s.erase(itl,itr);
        s.insert({l,r,v});
    }
    void perform(int l,int r)//对[l,r]进行操作
    {
        auto itr = spilt(r + 1),itl = spilt(l);
        for(;itl != itr;++itl)
        {
```

```
            //work

        }
    }
};
```

动态规划

数位dp

```
int dp[N][11],up[N];

// pos: 当前处理的数位
// pre: 上一个有效数字，10表示不存在
// li: 高位前缀是否等于上界
// le: 是否仍处于前导零阶段
int dfs(int pos,int pre,bool li,bool le)
{
    if(pos == 0)return !le;
    if(!li && !le && dp[pos][pre] != -1)return dp[pos][pre];
    int mx = li ? up[pos] : 9;
    int ans = 0;
    for(int d = 0;d <= mx;d++)
    {
        // 题目约束：不能有相邻相同数字
        if(!le && d == pre)continue;

        bool nlimit = li && (d == up[pos]);
        bool nlead = le && (d == 0);
        int npre = nlead ? 10 : d;
        ans += dfs(pos - 1,npre,nlimit,nlead);
    }
    if(!li && !le)dp[pos][pre] = ans;
    return ans;
}
// 统计 [1,n] 中满足要求的整数数量
int calc(int n)
{
    if(n <= 0)return 0;
    std::memset(dp,-1,sizeof(dp));
    int len = 0;
    while(n)
    {
        up[++len] = n % 10;
        n /= 10;
    }
    return dfs(len,10,1,1);
}
void solve()
{
    int l,r;
    std::cin >> l >> r;
```

```
    std::cout << calc(r) - calc(l - 1) << "\n";
}
//数字特别大时
int calc(string s)//
{
    memset(dp,-1,sizeof(dp));
    reverse(s.begin(),s.end());
    int len = 0;
    for(char c : s)
        up[++len] = c - '0';
    return dfs(len,10,10,1,0,0);
}
```